

CPPDescent

68d45e6 (with uncommitted changes)

Generated by Doxygen 1.9.6

1 CPP-Descent	1
1.1 Dependencies	1
1.1.1 Minor dependencies	1
1.2 Contributors	1
1.3 Bibliography	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 cppdescent Class Reference	7
4.1.1 Detailed Description	7
4.1.2 Member Function Documentation	7
4.1.2.1 hello()	7
4.2 List Class Reference	8
4.3 ListNode Class Reference	8
5 File Documentation	9
5.1 common.hpp	9
5.2 cppdescent.hpp	9
5.3 linkedList.hpp	9
5.4 cppdescent.cpp File Reference	10
Index	11

Chapter 1

CPP-Descent

Part of the "Software Development for Computing Systems" course of the Department of Informatics and Telecommunications, UoA for the first semester of the academic year 2023-2024.

A library that implements the KNN-Graph creation and search algorithms described in [1], [2].

1.1 Dependencies

For local development, you will need `Make` and `CMake`. To install them on an apt-based linux distro, run the following commands:

```
$ sudo apt update && sudo apt upgrade
$ sudo apt install make
$ sudo apt install cmake
```

1.1.1 Minor dependencies

- The project uses `Google Test` for its testing needs. Gtest is used as a git submodule of this repo, that is downloaded by CMake the first time you clone. That means that nothing extra needs to be installed to run the tests.
- If you would like to generate the documentation, you will need `doxygen` installed. ## Code Structure

In the `wiki`, you can find documentation generated by `Doxygen`. It is also available in `pdf form`.

1.2 Contributors

- `Konstantinos Chousos` (1115202000215)
- `Anastasios-Fedon Seitanidis` (1115202000179)

1.3 Bibliography

[1] W. Dong, C. Moses, and K. Li, "Efficient k-nearest neighbor graph construction for generic similarity measures," in *Proceedings of the 20th international conference on World wide web*, in WWW '11. New York, NY, USA: Association for Computing Machinery, Mar. 2011, pp. 577–586. doi: [10.1145/1963405.1963487](https://doi.org/10.1145/1963405.1963487).

[2] "How PyNNDescent works — pynndescent 0.5.0 documentation." Accessed: Oct. 10, 2023. [Online]. Available: https://pynndescent.readthedocs.io/en/latest/how_pynndescent_works.html

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

cppdescent	A sample function for the cppdescent class, used to test the correctness of the CMake files . . .	7
List		8
ListNode		8

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

common.hpp	9
cppdescent.hpp	9
linkedList.hpp	9
cppdescent.cpp	
Implementation of cppdescent library	10

Chapter 4

Class Documentation

4.1 cppdescent Class Reference

A sample function for the cppdescent class, used to test the correctness of the CMake files.

```
#include <cppdescent.hpp>
```

Public Member Functions

- int [hello](#) (void)
Prints "Hello World".

4.1.1 Detailed Description

A sample function for the cppdescent class, used to test the correctness of the CMake files.

4.1.2 Member Function Documentation

4.1.2.1 hello()

```
int cppdescent::hello (  
    void )
```

Prints "Hello World".

Returns

int exit code

The documentation for this class was generated from the following files:

- [cppdescent.hpp](#)
- [cppdescent.cpp](#)

4.2 List Class Reference

Public Member Functions

- **List** (DestroyFunc destroyValue)
- int **getSize** ()
- void **insertNext** ([ListNode](#) *node, Pointer value)
- void **removeNext** ([ListNode](#) *node)
- [ListNode](#) **find** ()
- int **setDestroyValue** (int value)
- [ListNode](#) **next** ([ListNode](#) node)
- Pointer **nodeValue** ([ListNode](#) node)
- [ListNode](#) **findNode** ([ListNode](#) node)
- int **increaseSize** ()
- int **decreaseSize** ()
- [ListNode](#) * **getHead** ()
- [ListNode](#) * **getTail** ()

The documentation for this class was generated from the following files:

- linkedList.hpp
- linkedList.cpp

4.3 ListNode Class Reference

Public Member Functions

- **ListNode** (Pointer value)
- int **setNext** ([ListNode](#) *next)
- [ListNode](#) * **getNext** ()
- Pointer **getValue** ()

The documentation for this class was generated from the following files:

- linkedList.hpp
- listNode.cpp

Chapter 5

File Documentation

5.1 common.hpp

```
00001 #pragma once
00002
00003 typedef void* Pointer;
00004
00005 typedef int (*CompareFunc)(Pointer a, Pointer b);
00006
00007 typedef void (*DestroyFunc)(Pointer value);
```

5.2 cppdescent.hpp

```
00001
00006 class cppdescent {
00007 public:
00013     int hello(void);
00014 };
```

5.3 linkedList.hpp

```
00001 #pragma once
00002 #include "common.hpp"
00003
00004 class ListNode {
00005 public:
00006     ListNode();
00007     ListNode(Pointer value) : value(value){};
00008     ~ListNode();
00009     int setNext(ListNode* next);
00010     ListNode* getNext();
00011     Pointer getValue();
00012
00013 private:
00014     ListNode* next;
00015     Pointer value;
00016 };
00017
00018 class List {
00019 public:
00020     List(DestroyFunc destroyValue);
00021     ~List();
00022     int getSize();
00023     void insertNext(ListNode* node, Pointer value);
00024     void removeNext(ListNode* node);
00025     ListNode find();
00026     int setDestroyValue(int value);
00027     ListNode next(ListNode node);
00028     Pointer nodeValue(ListNode node);
00029     ListNode findNode(ListNode node);
00030     int increaseSize();
00031     int decreaseSize();
```

```
00032     ListNode* getHead();
00033     ListNode* getTail();
00034
00035 private:
00036     ListNode* dummy;
00037     ListNode* head;
00038     ListNode* tail;
00039     int size;
00040     DestroyFunc destroyValue;
00041 };
```

5.4 cppdescent.cpp File Reference

Implementation of cppdescent library.

```
#include "cppdescent/cppdescent.hpp"
#include <iostream>
Include dependency graph for cppdescent.cpp:
```

Index

common.hpp, [9](#)
cppdescent, [7](#)
 hello, [7](#)
cppdescent.cpp, [10](#)
cppdescent.hpp, [9](#)

hello
 cppdescent, [7](#)

linkedList.hpp, [9](#)
List, [8](#)
ListNode, [8](#)